Digital circuit design (part 1)

M. Sonza Reorda

Politecnico di Torino
Dip. di Automatica e Informatica



1

1

Outline

- **Introduction**
- **Gate-level design**
- **RT-level design**
- **System-level design**

2

2

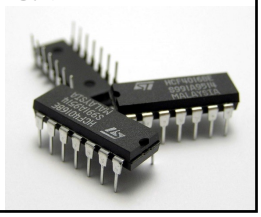
Introduction

A *system* is a collection of objects (*components*) connected to form a homogeneous entity with a well-defined functionality.

The function performed by the system is determined by:

- functions performed by the components
- how the components are connected (*topology*).

A *circuit* is a particular type of system.



3

3

Digital systems

Computer systems will be considered, which act on a set of input data A and produce a set of output data B .

It is assumed that the data in A and B are discrete quantities (*digital system*) corresponding to values represented through bits.

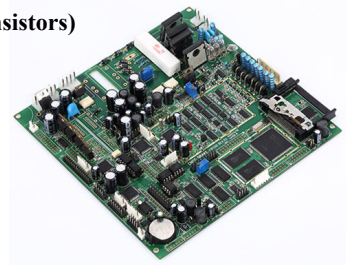
4

4

Examples of digital systems

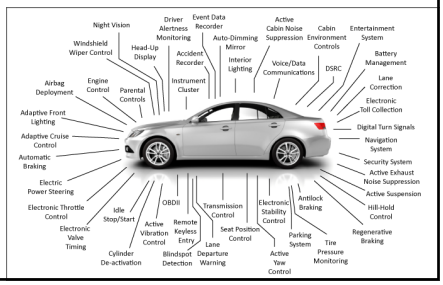


- Simple digital circuit (hundreds or thousands of transistors)
  - Counter / Adder
- Complex digital circuit (millions of transistors)
  - Microcontroller / Memory
- System on Chip or SoC (billions of transistors)
- Electronic board
- Complex processing system
  - Car
- Data Center







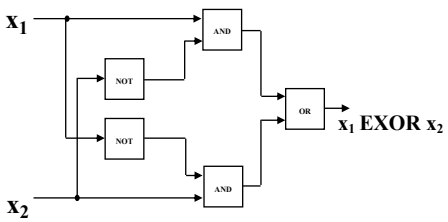


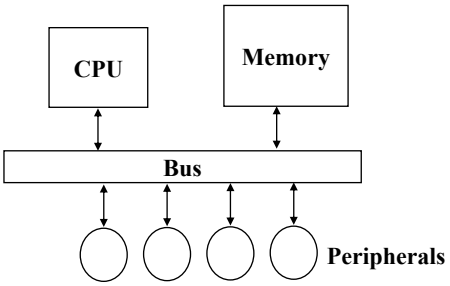
5

Block diagram

It allows us to easily describe the structure of a system by modeling it with a *graph*:

- vertices ⇒ components
- arcs ⇒ lines for data transfer between components.





6

6

Structure / behavior

Structure **= Block diagram without functional information**

Behavior **= Transformation function from input A to output B.**

7

Design

Let consider:

- **a system to be implemented, whose *behavior* is known**
- **a number of *specifications* (e.g., in terms of cost)**
- **a set of *components* (usually already available), whose behavior and characteristics are known.**

The design activity consists in identifying a suitable subset of the available components and a connection of such components so that the overall behavior is the desired one the specifications (including the cost) are met.

8

Specifications

They may include specific constraints on parameters such as

- speed
- power consumption
- reliability
- duration
- size
- etc.

9

9

Cost

The cost you try to minimize in your design is usually a weighted combination of different costs, including

-The design cost

-The production cost

-The maintenance cost.

10

10

System model

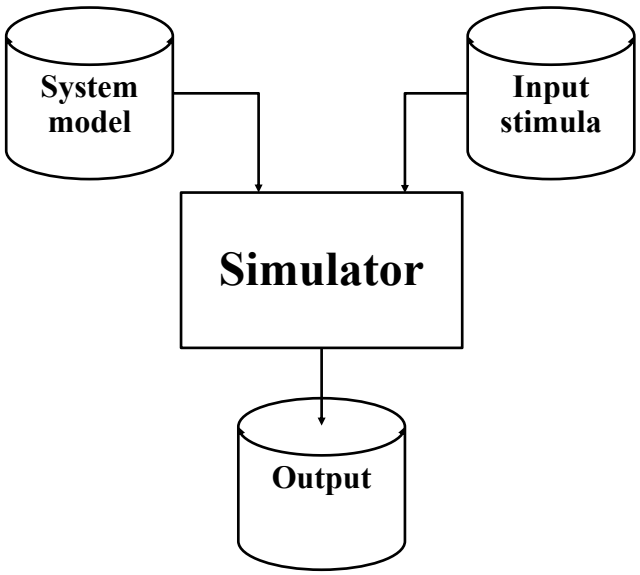
The design activity is strictly dependent on modeling, through which what you want to accomplish is formally described.

Using the model it is often possible to perform a series of checks (e.g., by simulation) before physical realization.

11

11

Simulation



12

12

Combinational and sequential systems

A system can be:

- ***Combinational*, if the values of its outputs depend solely on the values applied to its inputs at that instant.**

  Example: adder

- ***Sequential*, if the output values depend on both the current input values and the values applied in the previous instances.**

  Example: Counter

13

13

Combinational systems

The value of the outputs at a given moment depends solely on the input values at that instant.

The behavior of a combinational system can be described through

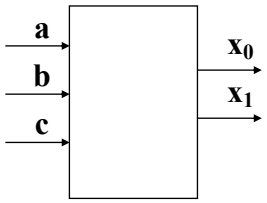
- **a *truth table*, which specifies for each input combination the corresponding output combination, or**
- **the *Boolean function* implemented by the outputs.**

14

14

Example

Let's consider the system that implements a 3-bit priority encoder, which can generate on the output x the index of the input bit equal to 1 having the maximum index:




```
if(a==1)
      x=3;
      else if(b==1)
            x=2;
            else if(c==1)
                  x=1;
                  else
                        x=0;
```

15

Example (II)

The system behavior can be described as a truth table or as a Boolean function:

a	*b*	*c*	x_1	x_0
0	0	0	0	0
0	0	1	0	1
0	1	0	1	0
0	1	1	1	0
1	0	0	1	1
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

Truth table

Boolean function

$$x_1 = a + b$$

$$x_0 = a + \bar{b}c$$

16

Sequential systems

The response of a sequential system at any time depends on

- the values X applied at that time to the system inputs
- The past history of the system itself, which is usually encoded by the state variable Y of the system.

Generally, the state of the system is not directly observable from the outside.

17

17

Description formats

The description of a sequential system can take three forms:

- states / outputs table
- states / output diagram
- transition state function and output function.

18

18

Example

Let consider a system having one input I and one output O.

The system samples the input I with a given frequency.

The output O holds the value 1 iff during the last 3 sampling times the input I has hold the values 101, the value 0 otherwise.

19

19

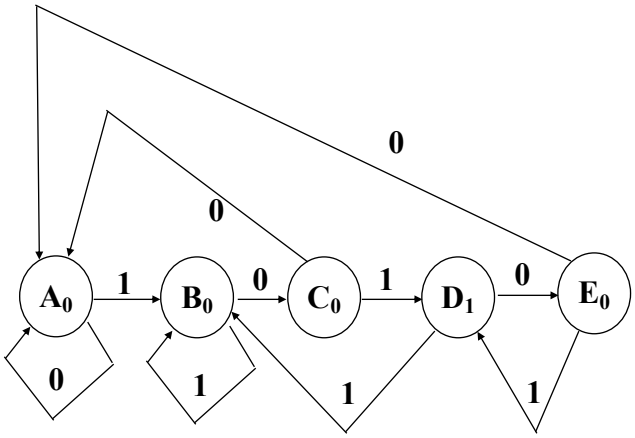
State variable

- It has to store the history of the system
- Hence, it could store the values in the last 3 sampling times (i.e., 3 bits)
- Hence, the status variable can assume 8 possible values
- It is possible to demonstrate that 5 values are sufficient.

20

20

State / output diagram



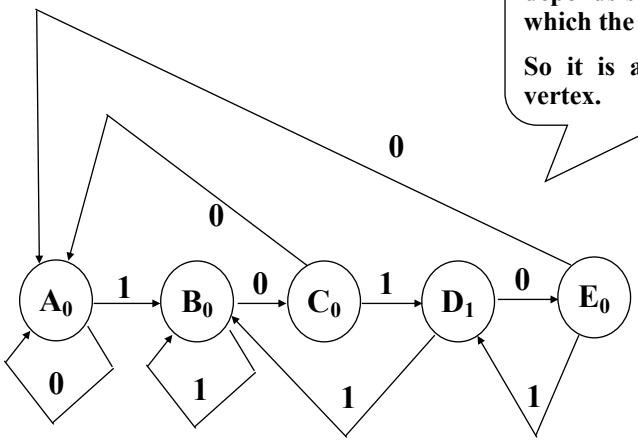
21

21

State / output diagram

In this case the output depends solely on the state in which the system is located.

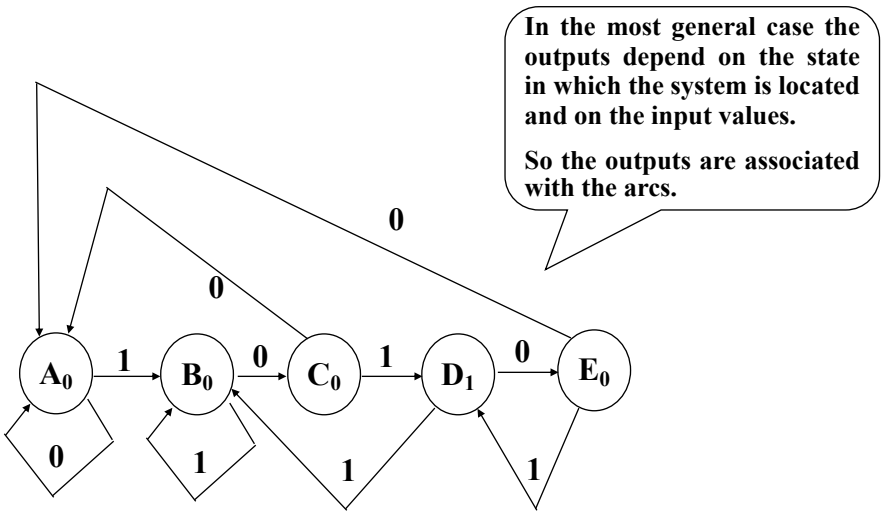
So it is associated with the vertex.



22

22

State / output diagram



23

23

State / output table

Current state	Input	Future state	Output
A	0	A	0
A	1	B	0
B	0	C	0
B	1	B	0
C	0	A	0
C	1	D	0
D	0	E	1
D	1	B	1
E	0	A	0
E	1	D	0

24

24

Transition function

Let X, Y and Z be three spaces associated with input, status and output variables, respectively.

The transition function performs the following transformation

$$f: X \times Y \rightarrow Y \times Z$$

In order to express f as a Boolean function, it is necessary to perform the association between the symbols that identify the states and the respective Boolean representation.

25

25

Example: transition function in the symbolic version

f(0,A) → A,0

f(1,A) → B,0

f(0,B) → C,0

f(1,B) → B,0

f(0,C) → A,0

f(1,C) → D,0

f(0,D) → E,1

f(1,D) → B,1

f(0,E) → A,0

f(1,E) → D,0

26

26

Hierarchical design

Circuit design is sometimes done following a *top-down* approach.

In other cases, top-down techniques are combined with *bottom-up* ones.

Guidelines:

- components can be physical entities (integrated circuits, tabs, etc.) or predefined modules (e.g., belonging to a library)
- each component must be as independent as possible, so that it can be developed or modified independently
- in many cases the designer knows the functions and features of a module without knowing the details of its implementation (*black box*).

27

27

Design levels

The design of a circuit can be performed at different levels of abstraction.

At each level, a certain set of components are available.

A component at a certain level corresponds to a set of lower level components.

The design of a circuit can take place at different levels:

- electrical
- transistor (or switch)
- gates
- registers (or register transfer level, or RTL)
- system.

28

28

Design flow

The design of a circuit normally takes place by iterating at the different levels the same operations:

- definition of *specifications* (through appropriate formalism)
- *synthesis* (manual or automatic)
- *verification* (through simulation or otherwise).

If you follow the top-down approach, the project result at one layer often directly represents the set of lower level specifications.

29

29

Electrical level

The system is modeled as an interconnection of components such as resistors, capacitors, inductors, etc. connected in series or parallel.

Calculation of current and voltage values at different points (steady-state or transient) involves resolution of a differential equation system.

On the market there are several SW packages (e.g., *SPICE*) that can simulate small-sized circuits.

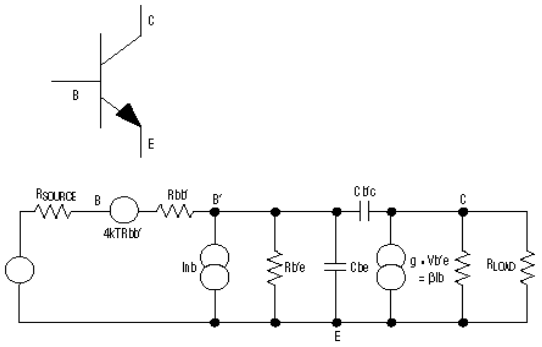
It is up to the designer to find a satisfactory trade-off between precision and computation time.

30

30

Example

You can model the behavior of a transistor through appropriate models, including the following (known as the *Giacoletto model*):



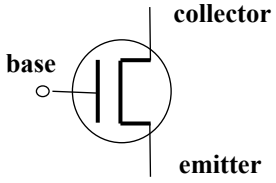
31

Transistor level

The system is seen as an interconnection of transistors, each considered as a voltage-controlled switch.

Depending on the technology used (PMOS or NMOS), the transistor is closed / open when the base signal has a voltage which is lower / higher of a certain threshold.

In any case, the switching speed of a transistor is in the order of nanoseconds.



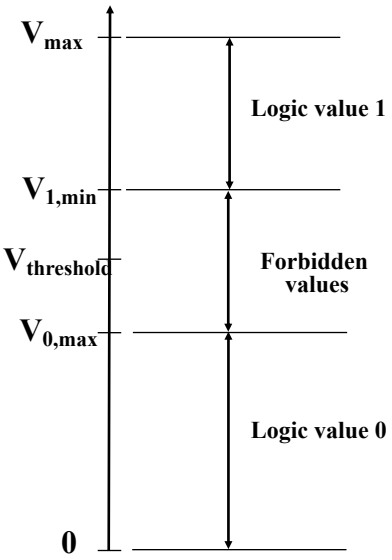
32

Voltages/logical values

In an electronic circuit it is common to associate the logic values (0/1) to voltages (high / low).

This operation requires to fix

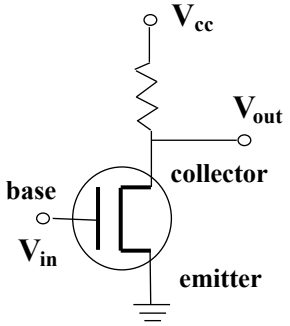
- a threshold to discriminate between high and low voltages
- a band of forbidden voltages, to avoid the effects of noise.



33

33

Inverter NMOS



If $V_{in}<V_{th}$:

$V_{out}=V_{cc}$

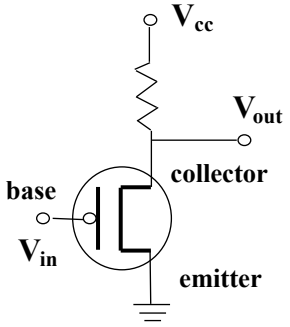
If $V_{in}>V_{th}$:

$V_{out}=V_{ground}$

34

34

Buffer PMOS



If $V_{in}>V_{th}$:

$V_{out}=V_{cc}$

If $V_{in}<V_{th}$:

$V_{out}=V_{ground}$

35

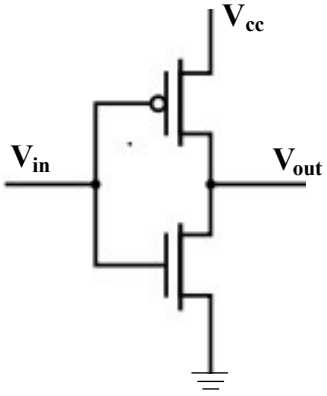
35

Inverter CMOS

It integrates NMOS technology with PMOS technology.

There is always one open and one closed transistor.

In this way there is never a connection between power and mass, and this reduces power consumption.



36

36

Logic gates

Logic gates implement the basic Boolean functions.

By using transistors, you can implement any logic gate.

You can then build a library of logic gates that can be used for the design at the upper level.

37

37

Gate level

At this level the components are the logic gates.

For this reason it is also known as a *logic level.*

Logic gates work on binary variables that can assume the two values 0 and 1.

The information processed at this level correspond to binary signals.

Ideally, each circuit line can assume only 2 voltage values corresponding to the 2 logic values 0 and 1.

38

38

Logic gates (I)

They are the building blocks for combinational circuits.

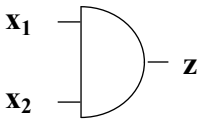
Each gate implements a Boolean function.

The set of gates normally consists of the following:

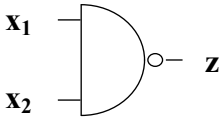
Truth table

0 0	0
0 1	0
1 0	0
1 1	1

AND



NAND



$Z = X_1 X_2 = X_1 \wedge X_2$ $\qquad$ $Z = \overline{x_1 x_2} = \overline{X_1 \wedge X_2}$

39

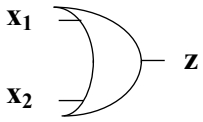
39

Logic gates (II)

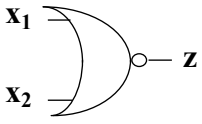
Truth table

0 0	0
0 1	1
1 0	1
1 1	1

OR



NOR



$Z = X_1 + X_2 = X_1 \vee X_2$ $\qquad$ $Z = \overline{X_1 + X_2} = \overline{X_1 \vee X_2}$

40

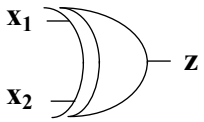
40

Logic gates (III)

Truth table

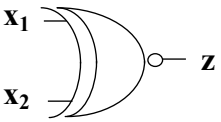
0 0	0
0 1	1
1 0	1
1 1	0

EXOR



$Z = X_1 \oplus X_2$

EXNOR

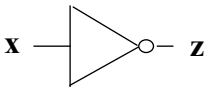


$Z = \overline{X_1 \oplus X_2}$

41

Logic gates (IV)

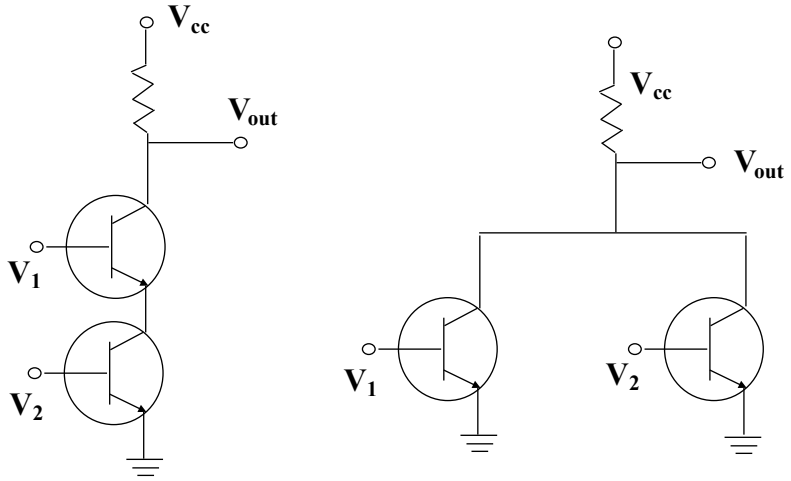
NOT



$Z = \overline{X}$

42

Implementing gates via NMOS transistors



NAND NOR

43

Complete set of gates

A set of gates is said to be *complete* if using the types of gates contained in the set, only, you can implement any combinational function.

The following are complete sets:

- **{NAND}**
- **{NOR}**
- **{AND, NOT}**
- **{OR, NOT}**
- **{AND, OR, NOT}**

In practice, circuits that implement a combinational function are frequently using only NAND (or NOR) gates.

44

Combinational circuits

They implement a combinational function.

A *combinational function* is a transformation

$$z: B^n \rightarrow B,$$

where B={0,1}.

Combinational functions do not involve time (unlike sequential ones) and can be described (if the number of variables is small) through truth tables.

Combinational circuits can be implemented through appropriate combinations of logic gates.

45

45

Well-formed combinational circuits

Well-formed Combinational Circuits (WFCCs) are combinational circuits that meet the following rules:

- a single line or a single gate is a WFCC
- the combination of 2 WFCCs is a WFCC
- if C1 and C2 are two WFCCs, the circuit obtained by connecting a set of C1 output lines to a set of C2 input lines is a WFCC
- if x_i and x_j are two input lines to a WFCC, the circuit obtained by connecting together x_i and x_j is a WFCC
- there are no cycles.

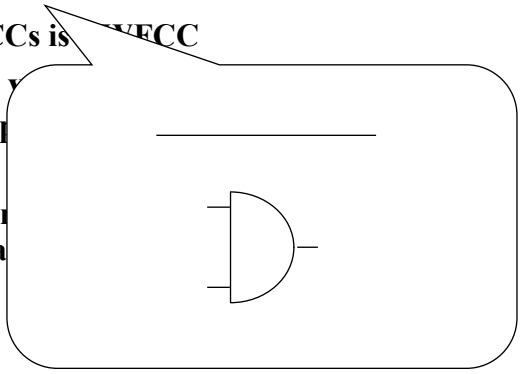
46

46

Well-formed combinational circuits

Well-formed Combinational Circuits (WFCCs) are combinational circuits that meet the following rules:

- **a single line or a single gate is a WFCC**
- **the combination of 2 WFCCs is a WFCC**
- **if C1 and C2 are two WFCCs, so is the circuit obtained by connecting a set of C1 outputs to a set of C2 inputs, to form a WFCC**
- **if x_i and x_j are two input lines, so is the circuit obtained by connecting together x_i and x_j**
- **there are no cycles.**



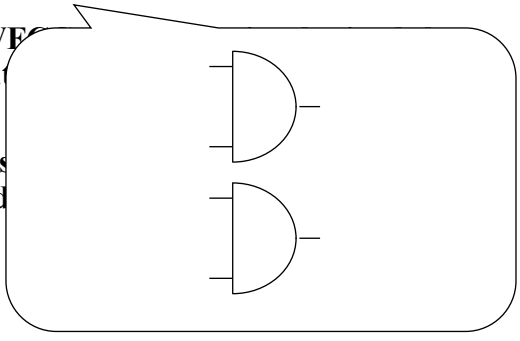
47

47

Well-formed combinational circuits

Well-formed Combinational Circuits (WFCCs) are combinational circuits that meet the following rules:

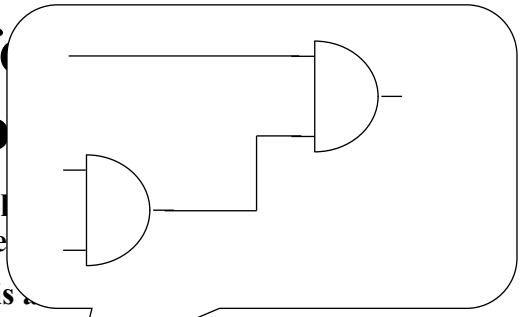
- **a single line or a single gate is a WFCC**
- **the combination of 2 WFCCs is a WFCC**
- **if C1 and C2 are two WFCCs, so is the circuit obtained by connecting a set of C1 outputs to a set of C2 inputs, to form a WFCC**
- **if x_i and x_j are two input lines, so is the circuit obtained by connecting together x_i and x_j**
- **there are no cycles.**



48

48

Well-f
combinatio



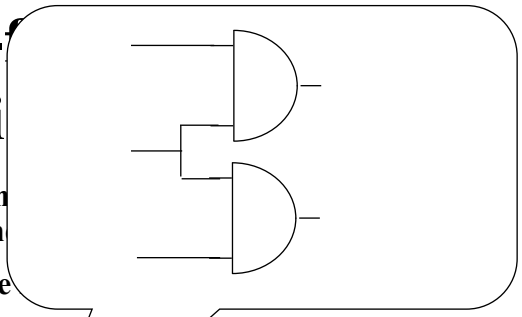
Well-formed Combinational combinational circuits that mee

- **a single line or a single gate is a**
- **the combination of 2 WFCCs is a WFCC**
- **if C1 and C2 are two WFCCs, the circuit obtained by connecting a set of C1 output lines to a set of C2 input lines is a WFCC**
- **if x_i and x_j are two input lines to a WFCC, the circuit obtained by connecting together x_i and x_j is a WFCC**
- **there are no cycles.**

49

49

Well-f
combinati



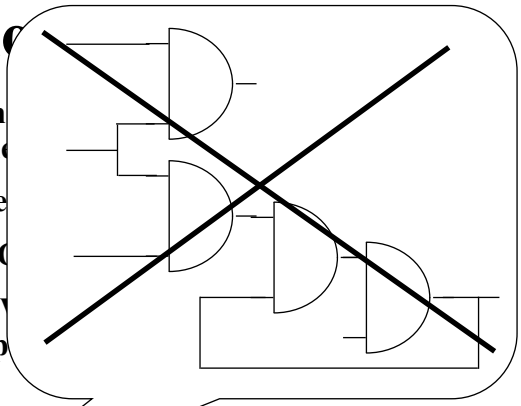
Well-formed Combination combinational circuits that m

- **a single line or a single gate**
- **the combination of 2 WFCCs is**
- **if C1 and C2 are two WF the circuit obtained by connecting a set of C1 output es to a set of C2 input lines is a WFCC**
- **if x_i and x_j are two input lines to a WFCC, the circuit obtained by connecting together x_i and x_j is a WFCC**
- **there are no cycles.**

50

50

Well-formed combinatio

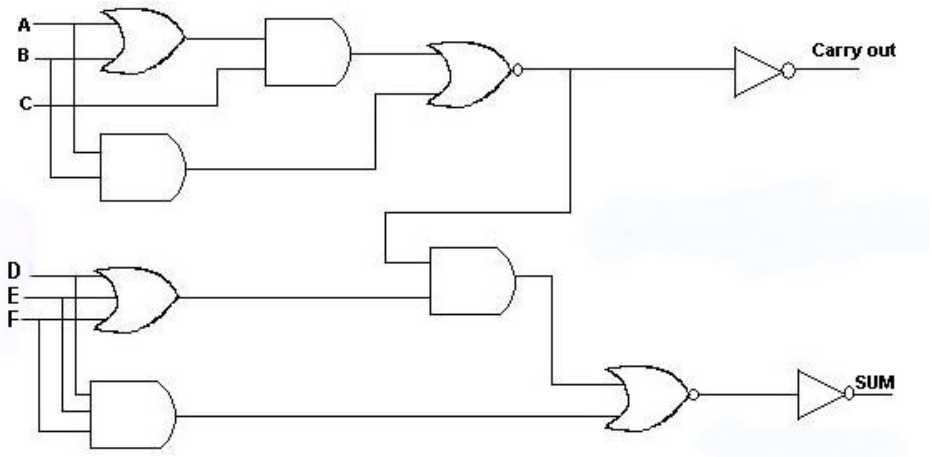


Well-formed Combination
combinational circuits that m

- a single line or a single gate
- the combination of 2 WFC
- if C1 and C2 are two
  connecting a set of C1 outp
  a WFCC
- if x_i and x_j are two input lines
  by connecting together x_i a
- there are no cycles.

51

Example of WFCC



52

Fanin

It is the number of input signals for a gate.

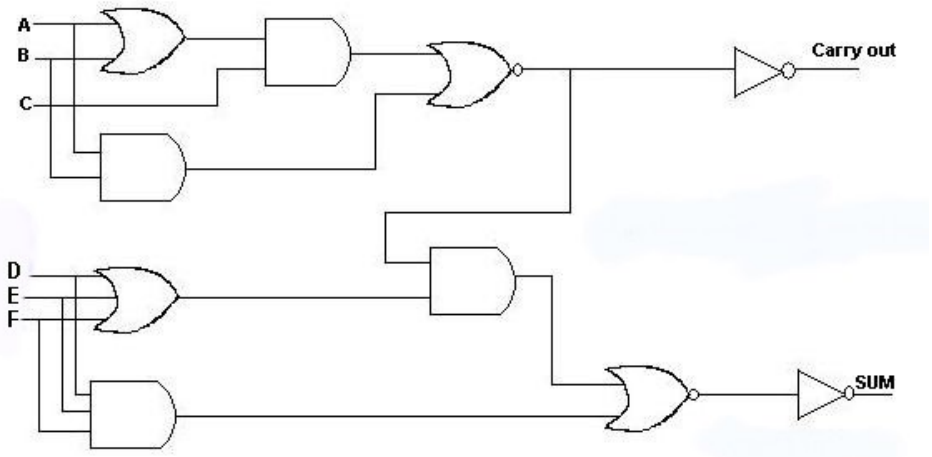
Fanout

It is the number of other gates driven by the output of a gate.

53

53

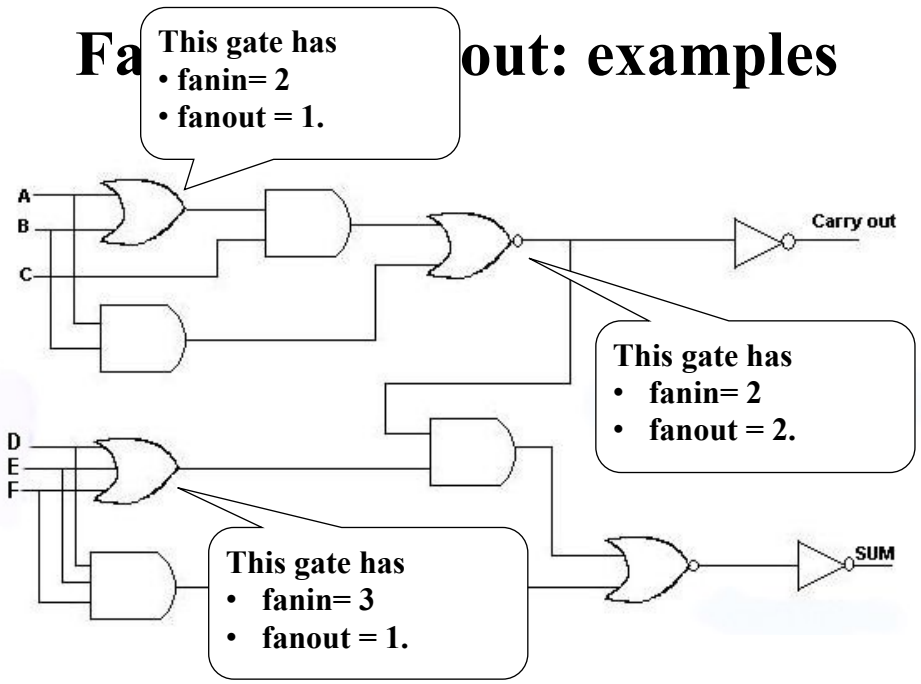
Fanin and fanout: examples



54

54

Fa out: examples



55

55

Design of a combinational circuit

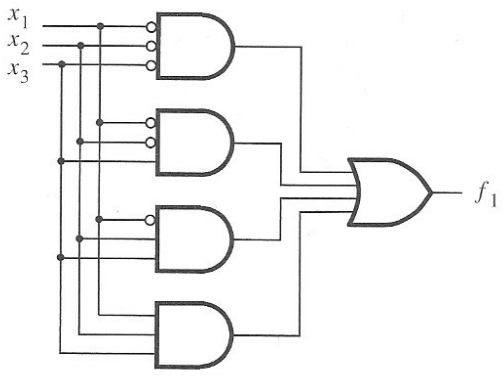
- It can be easily performed starting from the truth table
- For each row corresponding to an output value 1, an AND gate is inserted in the circuit
- The inputs of each AND gate are the circuit inputs, taken directly or negated depending on the left values in the row
- The outputs of all AND gates drive an OR gate.

56

56

Example (I)

x_1	x_2	x_3	f_1
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1



$$f_1 = \overline{x_1}\,\overline{x_2}\,\overline{x_3} + \overline{x_1}\,\overline{x_2}x_3 + \overline{x_1}x_2\,x_3 + x_1x_2x_3$$

57

Equivalence of logic circuits

It is possible that the same truth table corresponds to more than one circuit (and more than one Boolean expression).

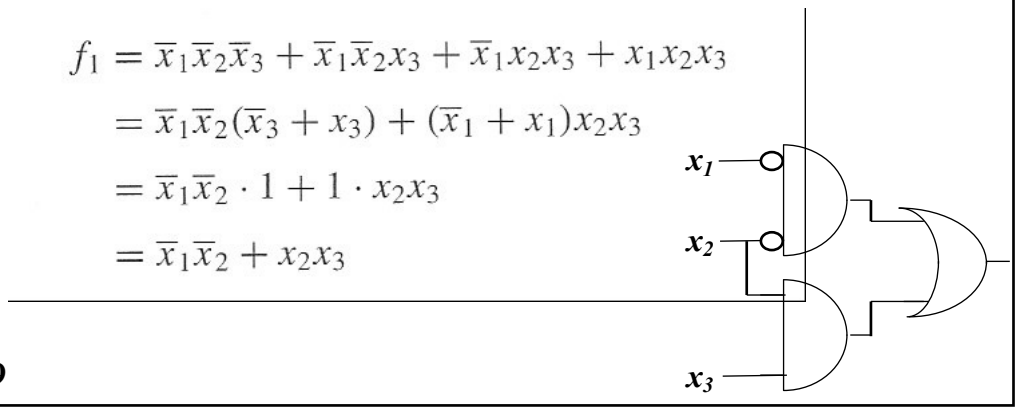
In this case, the behavior of the different circuits is the same and they are said to be *equivalent*.

58

Example (II)

The derivation of one form from the other can be obtained by applying the rules of Boolean algebra.

$$
\begin{aligned}
f_1 &= \overline{x}_1\overline{x}_2\overline{x}_3 + \overline{x}_1\overline{x}_2x_3 + \overline{x}_1x_2x_3 + x_1x_2x_3 \\
&= \overline{x}_1\overline{x}_2(\overline{x}_3 + x_3) + (\overline{x}_1 + x_1)x_2x_3 \\
&= \overline{x}_1\overline{x}_2 \cdot 1 + 1 \cdot x_2x_3 \\
&= \overline{x}_1\overline{x}_2 + x_2x_3
\end{aligned}
$$



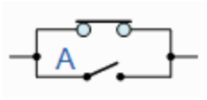
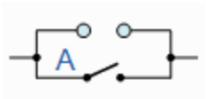
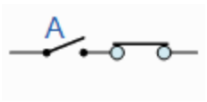
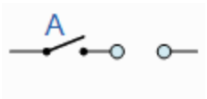
59

Boolean algebra

It can be defined through the following elements and operators:

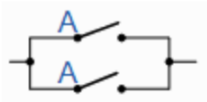
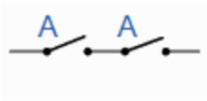
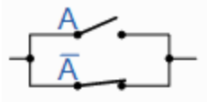
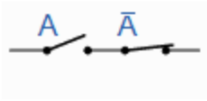
- ***elements*: 0, 1**
- ***operators*: AND (·), OR (+), NOT (¯)**
- ***axioms*:**

60

A + 1 = 1	A in parallel with closed = "CLOSED"		Annulment
A + 0 = A	A in parallel with open = "A"		Identity
A . 1 = A	A in series with closed = "A"		Identity
A . 0 = 0	A in series with open = "OPEN"		Annulment

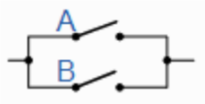
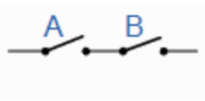
61

61

A + A = A	A in parallel with A = "A"		Idempotent
A . A = A	A in series with A = "A"		Idempotent
NOT $\overline{A}$ = A	NOT NOT A (double negative) = "A"		Double Negation
A + $\overline{A}$ = 1	A in parallel with NOT A = "CLOSED"		Complement
A . $\overline{A}$ = 0	A in series with NOT A = "OPEN"		Complement

62

62

A+B = B+A	A in parallel with B = B in parallel with A		Commutative
A.B = B.A	A in series with B = B in series with A		Commutative
$\overline{A+B} = \overline{A}.\overline{B}$	invert and replace OR with AND		de Morgan's Theorem
$\overline{A.B} = \overline{A}+\overline{B}$	invert and replace AND with OR		de Morgan's Theorem

63

63

Boolean algebra laws

- **L1:**

$$a+(b+c)=(a+b)+c$$
$$a(bc)=(ab)c$$

- **L2:**

$$a+a=a$$
$$aa=a$$

- **L3 (De Morgan):** $\overline{a+b} = \overline{a}\,\overline{b}$

$$\overline{ab} = \overline{a} + \overline{b}$$

- **L4:**

$$\overline{\overline{a}} = a$$

64

64

Minimization

When implementing the combinational circuit corresponding to a given truth table, it is preferable to identify the one with minimum cost, for example in terms of minimum number of gates.

65

65

Minimal combinational circuit

A combinational circuit is said to be *minimal* if:

- **It implements the Boolean function corresponding to the given truth table, and**
- **It meets certain constraints such as:**
  - **the depth of the circuit is less than a certain value**
  - **the number of gates used is minimal**
  - **the maximum fan is less than a certain value**
  - **the maximum fanout is less than a certain value.**

66

66

2-level circuits

A common solution to the problem of the design of minimal combinational circuits is represented by 2-level circuits.

The Quine-McCluskey method allows you to move from the truth table to a minimal 2-level circuit.

The method consists in transforming the truth table into a Boolean expression of either of the following two types:

- **Sum of products**

$$f(x_1, x_2, \ldots, x_n) = \sum_i \dot{x}_{i1}\, \dot{x}_{i2} \ldots \dot{x}_{in}$$

- **Product of sums**

$$f(x_1, x_2, \ldots, x_n) = \prod_i \dot{x}_{i1} + \dot{x}_{i2} + \ldots + \dot{x}_{in}$$

67

67

2-level circuits (II)

If all input variables are used (directly or complemented), these expressions can immediately be mapped into a 2-level circuit composed exclusively of AND and OR gates.

<u>Examples</u>

The function performed by a 2-bit adder can be written as

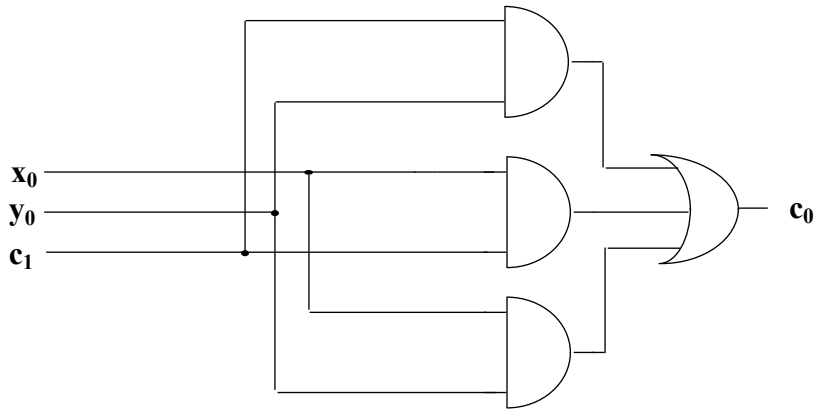
$$c_0 = x_0 c_1 + y_0 c_1 + x_0 y_0$$

which can be transformed into the following circuit:

68

68

2-level circuits (III)

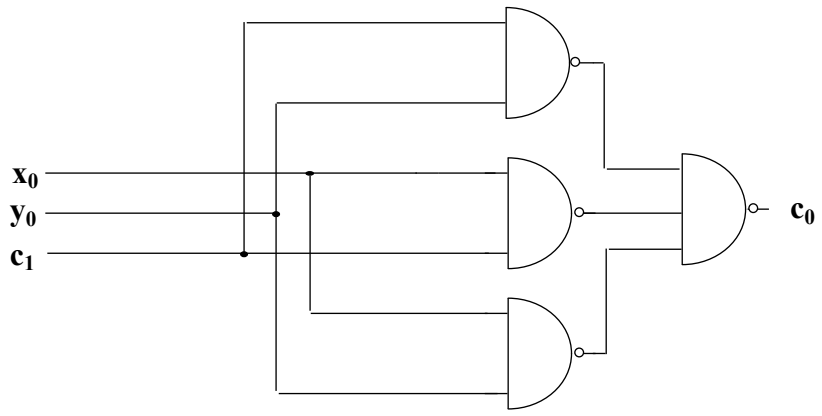


$c_0 = x_0c_1 + y_0c_1 + x_0y_0$

69

69

2-level circuits (IV)



Using the De Morgan law: $c_0 = \overline{\overline{x_0c_1} \bullet \overline{y_0c_1} \bullet \overline{x_0y_0}}$

70

70

Vocabulary

Literal $\dot{x}_i$

Minterm $\dot{x}_1 \dot{x}_2 \ldots \dot{x}_n$

Maxterm $\dot{x}_1 + \dot{x}_2 + \ldots + \dot{x}_n$

Cube $\dot{x}_i \ldots \dot{x}_j \ldots \dot{x}_k$

71

71

cabulary

It corresponds to x_i or $\overline{x_i}$

Literal $\dot{x}_i$

Minterm $\dot{x}_1 \dot{x}_2 \ldots \dot{x}_n$

Maxterm $\dot{x}_1 + \dot{x}_2 + \ldots + \dot{x}_n$

Cube $\dot{x}_i \ldots \dot{x}_j \ldots \dot{x}_k$

72

72

Canonicity

Any Boolean function has two possible canonical forms:

- as a sum of products, where all products are cubes
- as a product of sums.

73

Minimal circuit design

A combinational circuit is said to be *minimal* if one of the following conditions is true (equivalent to each other):

- the circuit consists of the least possible number of logic gates
- the circuit implements a function expressed in the sum of products form in which
  - the number of products is minimal, and
  - no literal can be deleted from a product without changing the function.

Similar conditions apply to the product of sums form.

74

Minimal circuit design (II)

Designing the minimal combinational circuit from the specifications has NP complexity.

In practice, it can be solved with automatic tools (e.g., *ESPRESSO*), with significant computation times.

For small combinational circuits (up to 5 input variables), the method of the *Karnaugh Map* allows you to move from the truth table to the minimal circuit.

75

75

Karnaugh maps

- **A Karnaugh map is a graphical representation of the behavior of a combinational system**
- **This representation can be used for functions with only one output and a number of inputs up to 4**
- **Using the Karnaugh maps it is possible to identify the minimum Boolean function for the system, and hence to make the corresponding minimal circuit.**

76

76

Example

x_1	x_2	x_3	f_1
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

Truth table

x_3 \ x_1x_2	00	01	11	10
0	1	0	0	0
1	1	1	1	0

Karnaugh map

77

Example

Each 1 cell corresponds to minterm

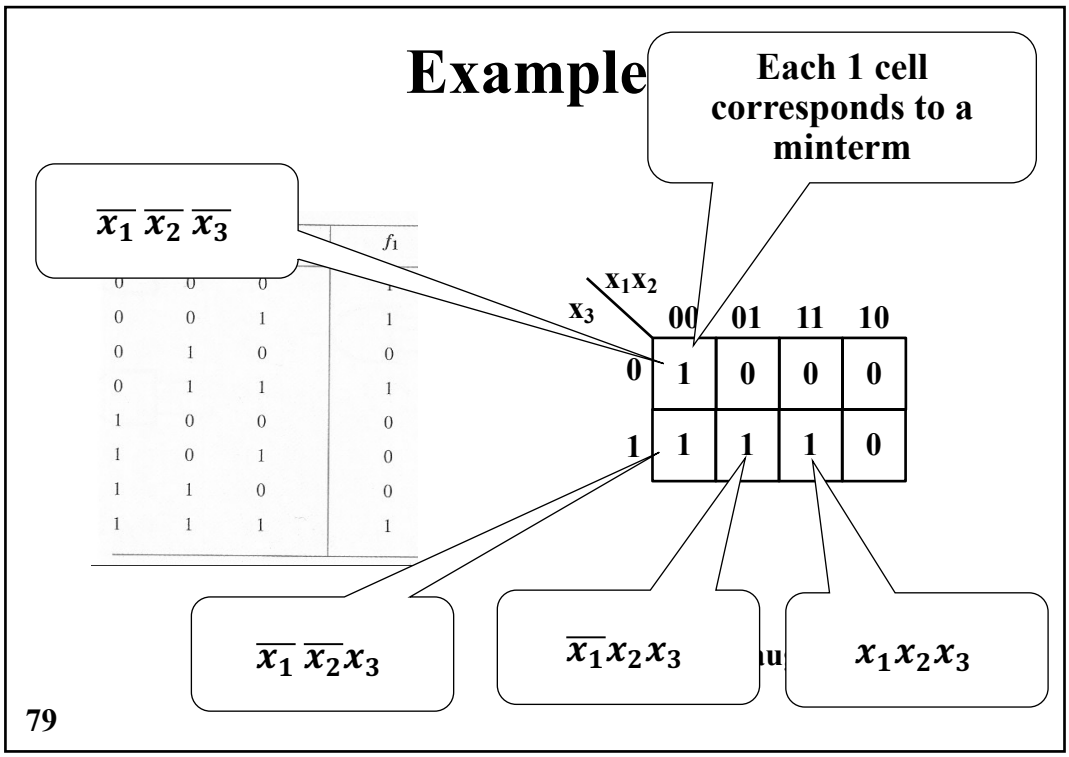
x_1	x_2	x_3	f_1
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

x_3 \ x_1x_2	00	01	11	10
0	1	0	0	0
1	1	1	1	0

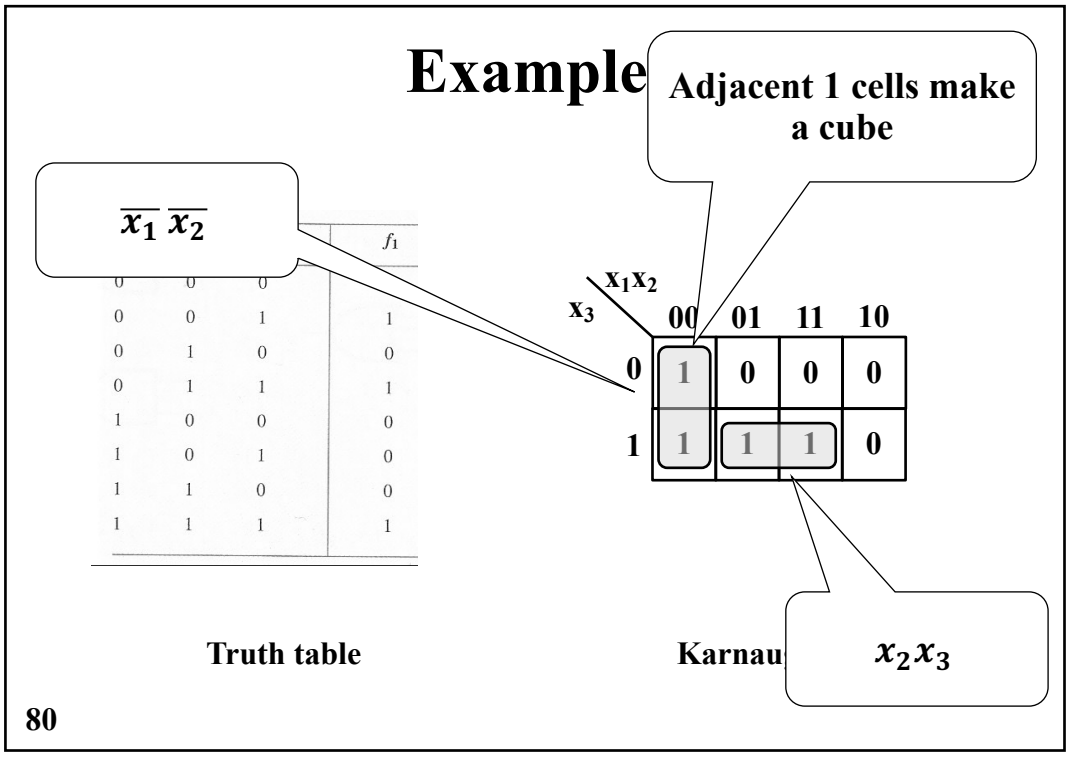
Adjacent 1 cells make a cube

The order of rows and columns does matter!

78


Example
Each 1 cell corresponds to a minterm
$\overline{x_1}\,\overline{x_2}\,\overline{x_3}$
f_1
x_1x_2
x_3
00 01 11 10
0 1 0 0 0
1 1 1 1 0
$\overline{x_1}\,\overline{x_2}x_3$
$\overline{x_1}x_2x_3$
$x_1x_2x_3$
79

79


Example
Adjacent 1 cells make a cube
$\overline{x_1}\,\overline{x_2}$
f_1
x_1x_2
x_3
00 01 11 10
0 1 0 0 0
1 1 1 1 0
Truth table
Karnau
x_2x_3
80

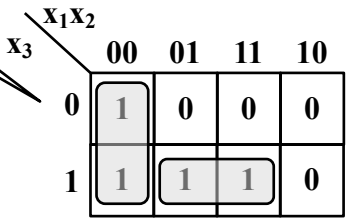
80

Example

$f_1 = \overline{x_1}\,\overline{x_2} + x_2x_3$

x_1	x_2	x_3	f_1
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

Truth table



Karnaugh map

81

Design procedure

Once you have built the Karnaugh map corresponding to the truth table, you have to

- Identify the minimum set of cubes that cover all 1s in the map of Karnaugh, choosing the ones of maximum size; cubes may eventually overlap
- Transform the cubes into the corresponding sum of products expression
- Build the corresponding circuit.

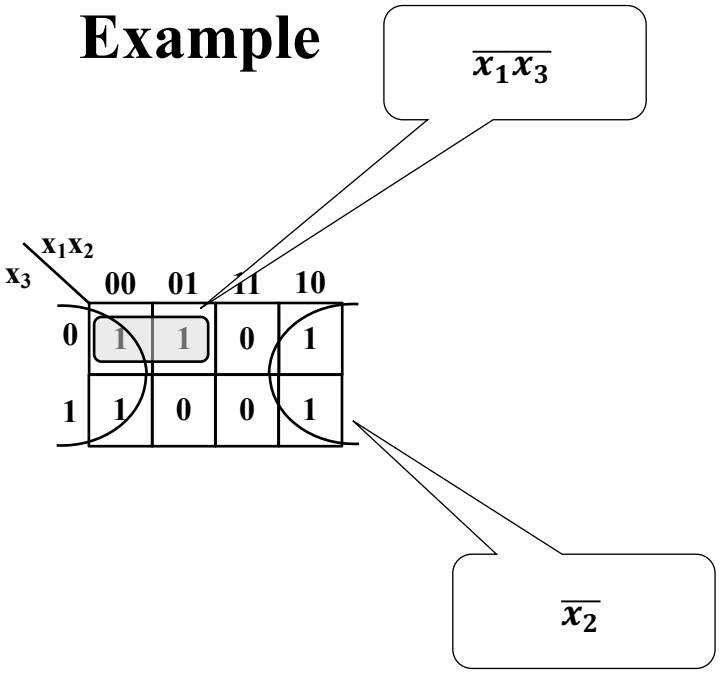
82

Example

x_3 \ x_1x_2	00	01	11	10
0	1	1	0	1
1	1	0	0	1

83

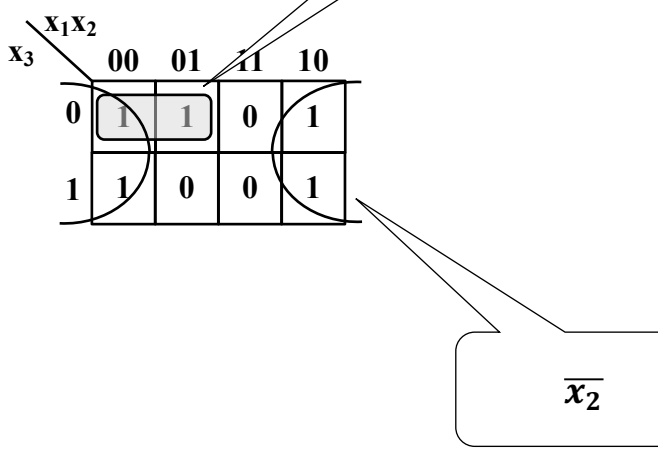
Example



84

Example

$f_1 = \overline{x_1}\,\overline{x_3} + \overline{x_2}$



$\overline{x_1\,x_3}$

$\overline{x_2}$

85

Karnaugh maps with 4 inputs

- They are composed of 16 cells
- In such maps, cubes with 1, 2, 4 or 8 elements can be identified.

86

Example

$x_3 x_4$ \ x_1x_2	00	01	11	10
00	0	0	1	1
01	0	0	0	0
11	1	0	0	1
10	0	0	1	1

87

87

Example

$f_1 = x_1\overline{x_4} + \overline{x_2}x_3x_4$

$x_1\overline{x_4}$

$x_3 x_4$ \ x_1x_2	00	01	11	10
00	0	0	1	1
01	0	0	0	0
11	1	0	0	1
10	0	0	1	1

$\overline{x_2}x_3x_4$

88

88

Example 2

$x_3\,x_4$ \ x_1x_2	00	01	11	10
00	0	1	1	0
01	1	1	1	1
11	1	1	1	1
10	0	0	0	0

89

89

Example 2

$f_1 = x_2\overline{x_3} + x_4$

$x_2\overline{x_3}$

$x_3\,x_4$ \ x_1x_2	00	01	11	10
00	0	1	1	0
01	1	1	1	1
11	1	1	1	1
10	0	0	0	0

x_4

90

90

Don’t care

In some cases the specifications for a combinational system may not have any output values at some input combinations, for example because they never appear.

In this case we talk about "don’t care" values, denoted with “-” or “X”.

The design process can exploit these values to minimize the circuit.

91

91

Example

Let us consider a function that receives the 4 bits corresponding to a BCD digit and returns the value 1 if and only if that digit corresponds to a multiple of 3:

Decimal value	#	Binary encoding b_3	b_2	b_1	b_0	f
0	0	0	0	0	0	0
1	1	0	0	0	1	0
2	2	0	0	1	0	0
3	3	0	0	1	1	1
4	4	0	1	0	0	0
5	5	0	1	0	1	0
6	6	0	1	1	0	1
7	7	0	1	1	1	0
8	8	1	0	0	0	0
9	9	1	0	0	1	1
Unused	10	1	0	1	0	x
	11	1	0	1	1	x
	12	1	1	0	0	x
	13	1	1	0	1	x
	14	1	1	1	0	x
	15	1	1	1	1	x

92

92

Example

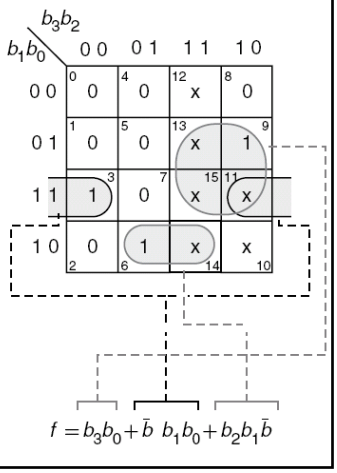
Let us consider a function that receives the 4 bits corresponding to a BCD digit and returns the value 1 if and only if that digit corresponds to a multiple of 3:

Decimal value	Binary encoding #	b_3	b_2	b_1	b_0	f
0	0	0	0	0	0	0
1	1	0	0	0	1	0
2	2	0	0	1	0	0
3	3	0	0	1	1	1
4	4	0	1	0	0	0
5	5	0	1	0	1	0
6	6	0	1	1	0	1
7	7	0	1	1	1	0
8	8	1	0	0	0	0
9	9	1	0	0	1	1
Unused	10	1	0	1	0	x
	11	1	0	1	1	x
	12	1	1	0	0	x
	13	1	1	0	1	x
	14	1	1	1	0	x
	15	1	1	1	1	x

93

Example

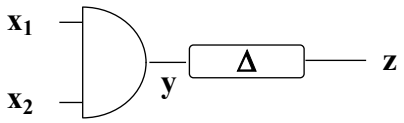
- **The corresponding Karnaugh map includes 6 boxes with the don't care value**
- **They can be considered as if they contain 0 or 1, depending on what is most convenient for the design.**



94

Sequential circuits

- They implement time-dependent functions
- They can store information
- They take advantage of gate delays:

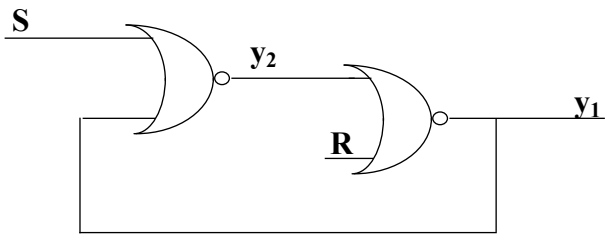


$z(t+\Delta)=x_1(t)x_2(t)$

95

Flip-Flop

It can store one bit for an indefinite time.

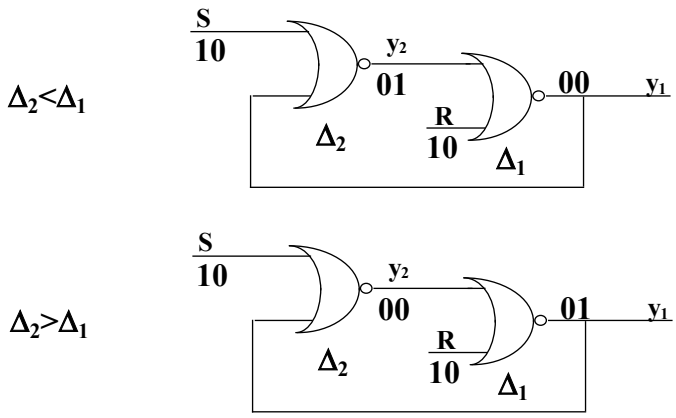


Behavior:

- S=R=0: it maintains the value on y_1
- S=0, R=1: it forces a 0 on y_1
- S=1, R=0: it forces a 1 on y_1
- S=1, R=1: forbidden configuration

96

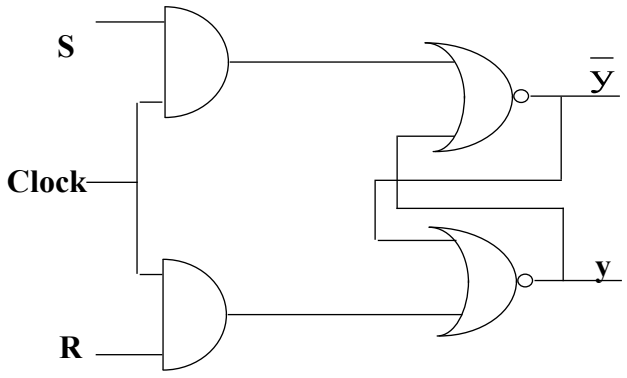
Transition S=R=1⇒ S=R=0



97

97

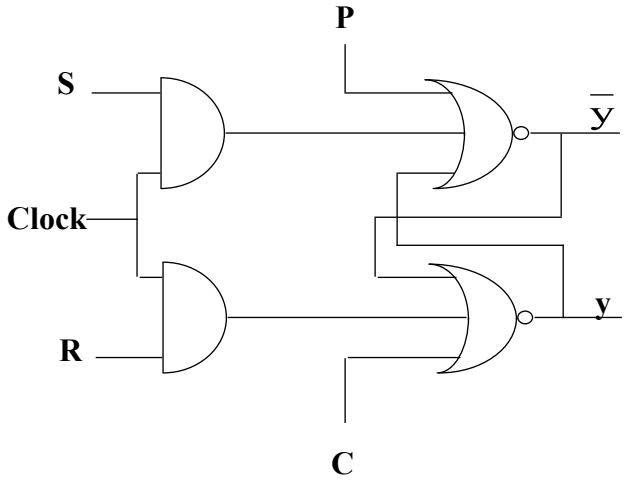
Synchronous Flip-Flop



98

98

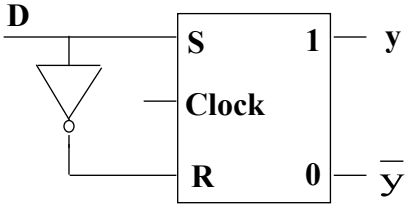
SR Flip-Flop with asynchronous Clock, Preset and Clear



99

99

D Flip-Flop



y \ D	0	1
0	0	1
1	0	1

State table

When Clock=1 the value on D is stored.

100

100

Synchronous circuits

The behavior of a sequential circuit depends on the relative values of gate delay in the combinational part (*races*).

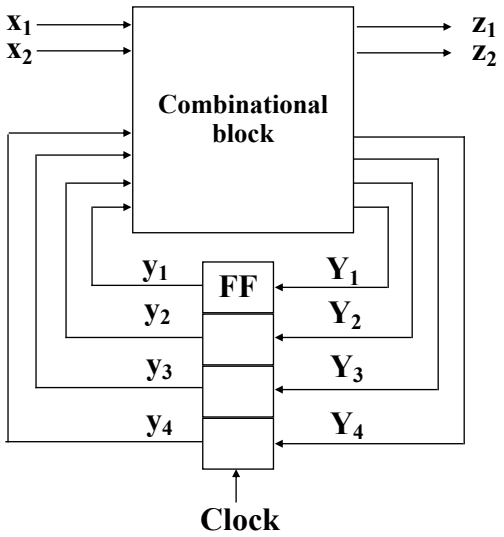
This can be avoided by storing the input values for all Flip-Flops at the same instant, determined by the value of the clock signal.

This signal has a square wave pattern with a certain frequency.

Clocked circuits are called *synchronous*.

101

Huffman's model



102

Operational frequency

To ensure the correct operation of a synchronous sequential circuit, the period of the clock signal must be higher than the maximum delay through the combinational block.

103

103

Finite state machine

- **A sequential system is also known as a *Finite State Machin*e (*FSM*)**
- **FSMs can be of two types:**
  - **Moore FSMs: The output value depends solely on the current value of the status variable**
  - **Mealy FSMs: The output value depends on the current value of the status variable and on the current value of the input.**

104

104

State table

A synchronous sequential circuit evolves through states determined by the values in the Flip-Flops.

Its behavior can thus be described through a state table, which describes the transitions between states depending on the value on the inputs.

105

Design of synchronous sequential circuits

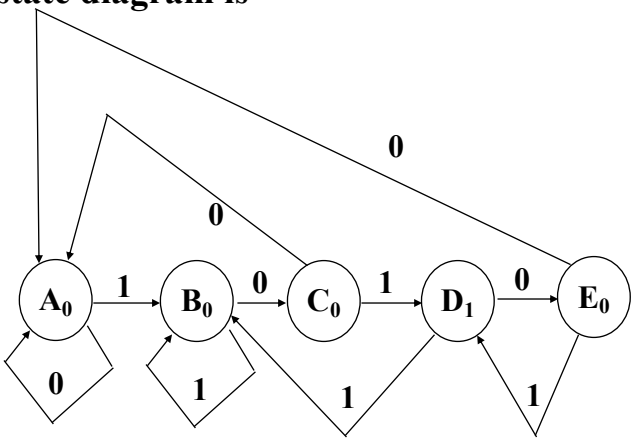
- Construction of the state table
- Minimization (elimination of equivalent states)
- State encoding
- Design of the Combinational Logic Truth Table
- Design of the Combinational Logic.

106

Example

Let consider the sequence recognizer for the sequence 1-0-1.

The state diagram is

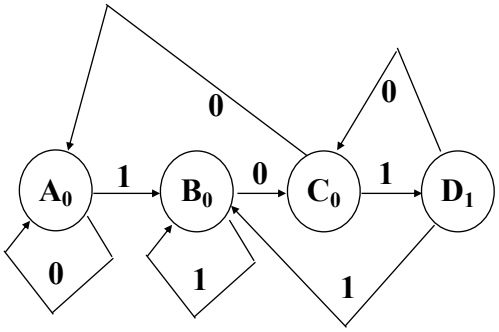


107

Example (II)

State E is equivalent to state C.

Hence, the diagram can be simplified:

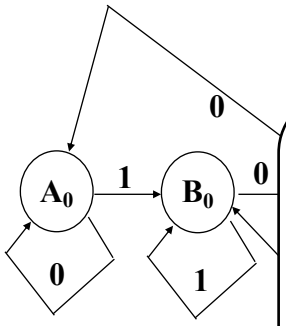


108

Example (II)

State E is equivalent to state C.

Hence, the diagram can be simplified:



Two state diagrams are *equivalent* if given any input sequence, their behavior in terms of output values is identical.

109

109

Example (III)

One possible state encoding is

	Y_1	Y_2
A	**0**	**0**
B	**0**	**1**
C	**1**	**0**
D	**1**	**1**

110

110

Example (IV)

The truth table of the state transition function is:

$i\ y_1\ y_2$	$\rightarrow Y_1\ Y_2$
0 00	→ 00
1 00	→ 01
0 01	→ 10
1 01	→ 01
0 10	→ 00
1 10	→ 11
0 11	→ 10
111	→ 01

The output value z can be easily generated by taking into account that it is 1 only when the circuit is in 11:

$i\ y1\ y2$	$\rightarrow z$
- 00	→ 0
- 01	→ 0
- 10	→ 0
- 11	→ 1

111

111

Example (V)

We can now synthesize the 3 functions Y1, Y2 and z, obtaining:

$$Y_1 = y_1\ \overline{y_2} i + y_2\ \bar{\imath}$$

$$Y_2 = i$$

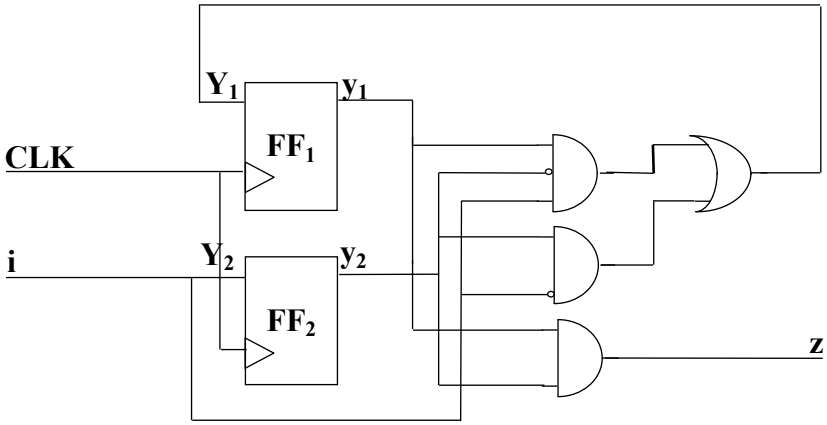
$$z = y_1 y_2$$

112

112

Example (VI)

The circuit implementing the sequence recognizer is the following:



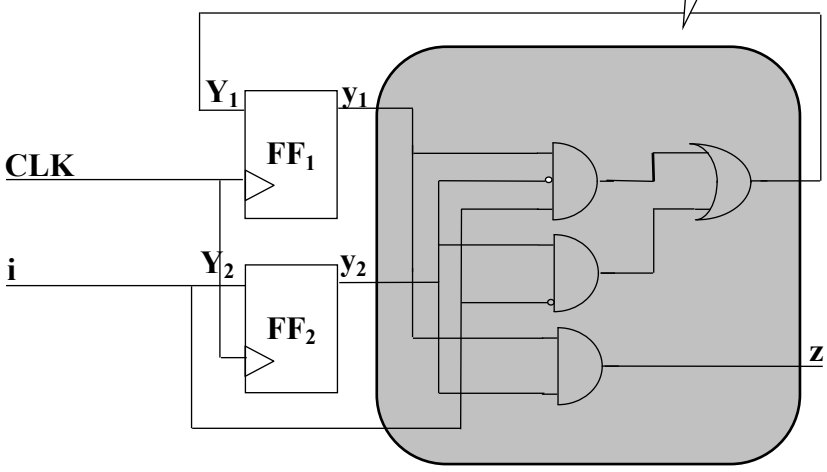
113

113

Example (VI)

Combinational block

The circuit implementing the sequence recognizer is the following:



114

114